

Part 8

ORM in Django

VERSION: 4.0.1

WRITTEN AND DESIGN BY: BABAR ALI

What is ORM?

One of the most powerful features of Django is its **Object-Relational Mapper (ORM)**, which enables you to interact with your database, like you would with SQL. In fact, Django's **ORM** is just a **Pythonical** way to create **SQL** to query and manipulate your **database** and get results in a **Pythonic** fashion.

The **ORM's** main goal is to transmit data between a relational database and application model. The ORM automates this transmission, such that the developer need not write any **SQL**. This makes the whole development process fast and error-free.

The Problem Solved by ORM

ORMs have certain benefits over the traditional approach. The main advantage **ORMs** provide is rapid development. **ORMs** make project more portable. It is easier to change the database if we use **ORMs**.

In the past, web developers needed to have knowledge of databases too. A database has been an important component from the start. The programming languages used for web development use classes and object for data-interpretation. The class is used to a defined data structure in web-applications. Then the same database schema is created in the database. This task requires skill and knowledge of **SQL**.

Queryset in Django

- ✓ We all use queries to retrieve data from the database. **Querysets** are Django's way to retrieve data from the database. The Django ORM lets us use **Querysets**.
- ✓ A **Queryset** is a list of objects of a model. We use **Querysets** to filter and arrange our data. These make our work as a Python developer easier.

What is Model in Django?

- ✓ A model is a built-in feature that Django uses to create tables, their fields, and various constraints. In short, Django model is the SQL of Database one uses with Django.
- ✓ SQL (Structured Query Language) is complex and involves a lot of different queries for creating, deleting, updating or any other stuff related to database.
- ✓ Django Models simplify the tasks and organize tables into models. Generally, each model maps to a single database table.

Django Model

- ✓ Each model is a Python class that subclasses **django.db.models.Model**.
- ✓ Each attribute of the model represents a database field.
- ✓ With all of this, Django gives you an automatically-generated database access API.

Example:

Name	Father Name	Roll No	Address
Babar Ali	Sammad Khan	145986	Shabqadar
Umair Khan	Haris Khan	345765	Halimzai
Imran Shah	Sadiq Shah	123578	Per Qilla

Create Your First Django Model

```
from django.db import models  
  
class Person(models.Model):  
    first_name = models.CharField(max_length=30)  
    last_name = models.CharField(max_length=30)
```

- ✓ **first_name** and **last_name** are fields of the model.
- ✓ Each field is specified as a class attribute, and each attribute maps to a database column.

Add Your App in Django Project

- ✓ If you are use your model first of all register your app in your Django model.
- ✓ Go to settings.py then Go to INSTALLED_APPS section in section and add:

```
INSTALLED_APPS = [  
    'first',  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
]
```


What is Migrate?

The migrate command looks at the INSTALLED_APPS setting and creates any necessary database tables according to the database settings in your `mysite/settings.py` file and the database migrations shipped with the app (we'll cover those later). You'll see a message for each migration it applies.

If you're interested, run the command-line client for your database and type `\dt` (**PostgreSQL**), `SHOW TABLES;` (**MariaDB**, **MySQL**), `.tables` (**SQLite**), or `SELECT TABLE_NAME FROM USER_TABLES;` (Oracle) to display the tables Django created.

Migrate Your Model

```
>>> python manage.py makemigrations
```



Migrations for 'first':
first\migrations\0001_initial.py
- Create model Person

```
>>> python manage.py migrate
```

Your Model Coding

Go to your project. Then Go to the app Go to Migration and open it:

```
CREATE TABLE myapp_person (  
  "id" NOT NULL PRIMARY KEY,  
  "first_name" (30) NOT NULL,  
  "last_name" (30) NOT NULL  
);
```

- ✓ “ID” is the **auto** field for any Django model. Every model have a **ID**.
- ✓ “**first_name**” is the **CharField** range is 30 characters and **NOT NULL**.

Playing With the API

Now, let's hop into the interactive Python shell and play around with the free API Django gives you. To invoke the Python shell, use this command:

```
>>> python manage.py shell
```

This command will help you to enter the Django API Shell.

Django Database Queries

```
>>> from first.models import Person
```

→ Import the Person Model

```
>>> Person.objects.all()
```

→ Get all the Records

```
>>> haris = Person(first_name="Haris", last_name="Khan")
```

→ Add record in Person table

```
>>> haris.save()
```

→ Save the record

```
>>> Person.objects.get(id=1)
```

→ Get a record where id is "1"

What is Django Admin.?

The Admin interface of a website is the control panel of your website/ project. It is used by the administrators of the website to add, delete and edit the content on the site, and also maintain other functionalities accordingly made by the developers.

The Django Admin is a fully featured interface and you won't need to write an Admin interface for your project now. To begin with, it is always better to know the process of setting up Django Admin site as you will need it in future projects.

Setup Django Admin Site

Below are the steps to use the Django Admin Site and implement user models:

Step 1.

After starting the Django server, if we search in the URL bar:

`localhost:8000/admin/`

Step 2.

Open the PowerShell or terminal and break the server by Ctrl + C.

Then type in this command:

```
>>> python manage.py createsuperuser
```

Continue...

Step 3:

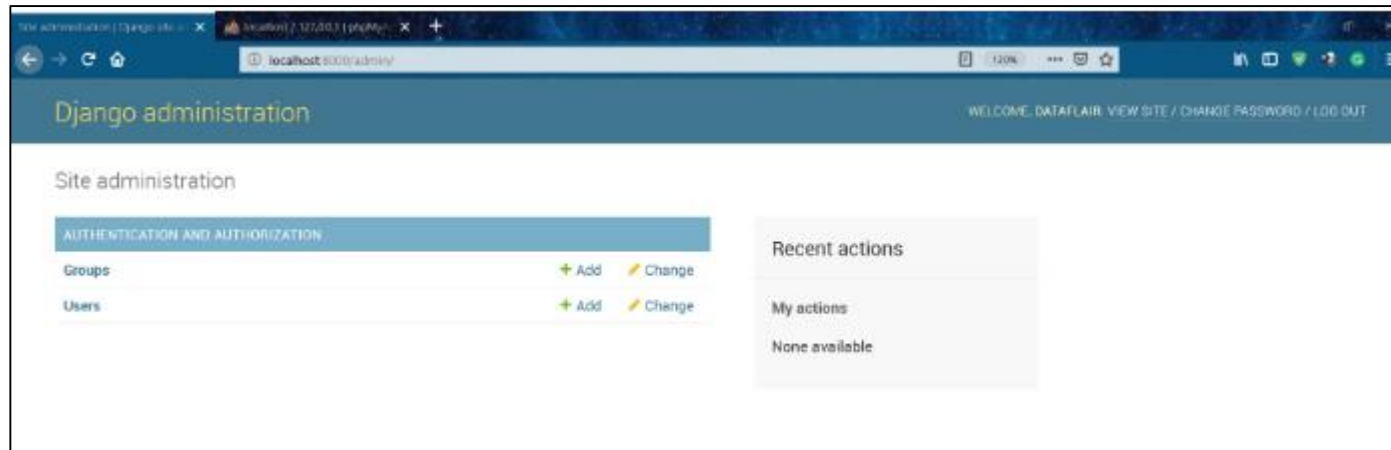
After executing this command, it will ask for your username, which is your choice again. Then it will ask you for the Email Id and Password. Complete the details.

There are some inbuilt security features here like if your password is totally numeric it will display a message to create a complex password and many such security loopholes have been covered up for users.

Continue...

Step 4.

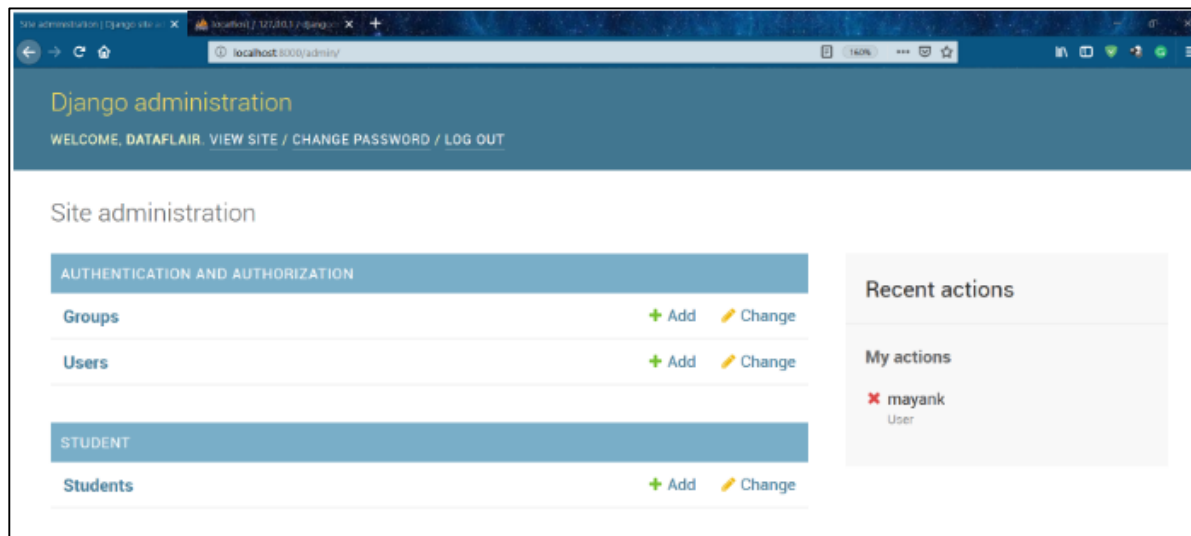
Now login in admin using the credentials you just created.



Continue...

Step 5.

Now click on the users field and you should see something like this.



Continue...

Step 6:

Now, you can check whether the user you created got saved in it. Although you didn't do anything with the database.

Notice, that there are many tables which we didn't create but are inside our database. They are all **django.contrib** package's imported models tables. **django.contrib** is a package for Django which is comparable to the relation of standard library with Python. Almost all the functionality we need is already on the django.contrib package.

Continue...

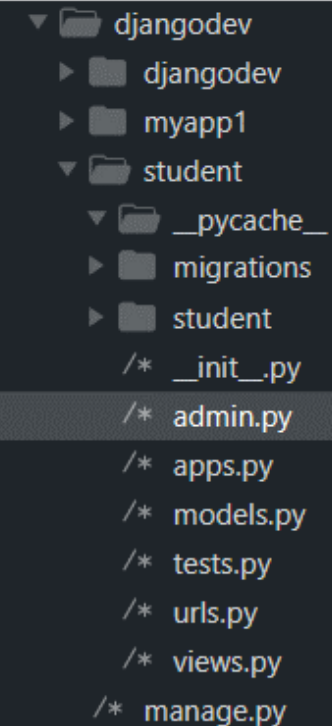
Step 9:

Now open the file `admin.py` of your app. The directory structure should be like this.

Step 10:

Open the `admin.py` file and type in these two lines for register model

```
from .models import Student
admin.site.register(Student)
```



```
▼ djangodev
  ► djangodev
  ► myapp1
  ▼ student
    ▼ __pycache__
    ► migrations
    ► student
    /* __init__.py
    /* admin.py
    /* apps.py
    /* models.py
    /* tests.py
    /* urls.py
    /* views.py
    /* manage.py
```

What is Super User.?

It is basically the credentials of the admin for your website, which can do any data manipulation and content manipulation related to your website.

Also, note that you should never share the superuser credentials with anyone, as it is the main controller of your website. When the website goes live, it is vital information for maintaining and controlling your website.

After successfully creating the superuser we are now going to log in to that admin page to start your Django server again.

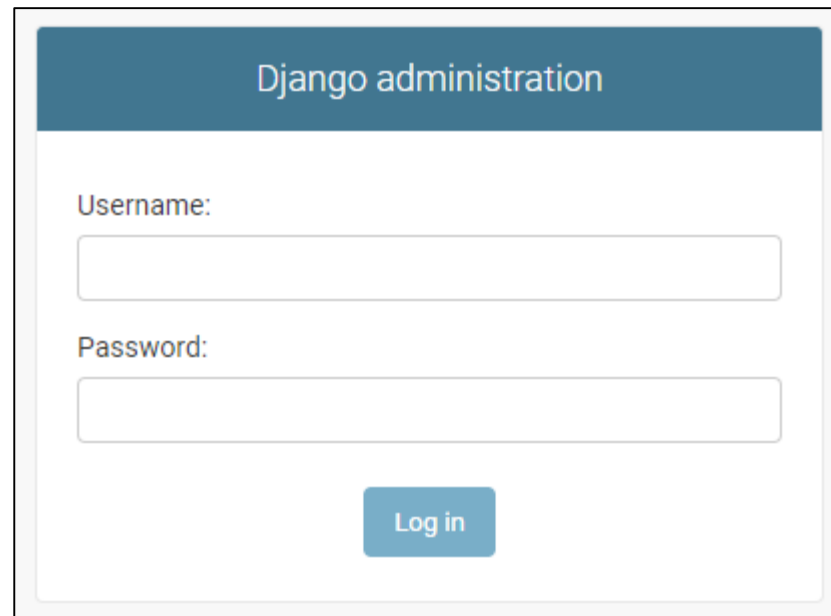
Create Super User

```
>>> python manage.py createsuperuser
```

- ✓ Enter “**username**” and “**Email**”.
- ✓ Enter “**password**” and write “**Y**” for creating a super user.
- ✓ Your Super User Created Successfully

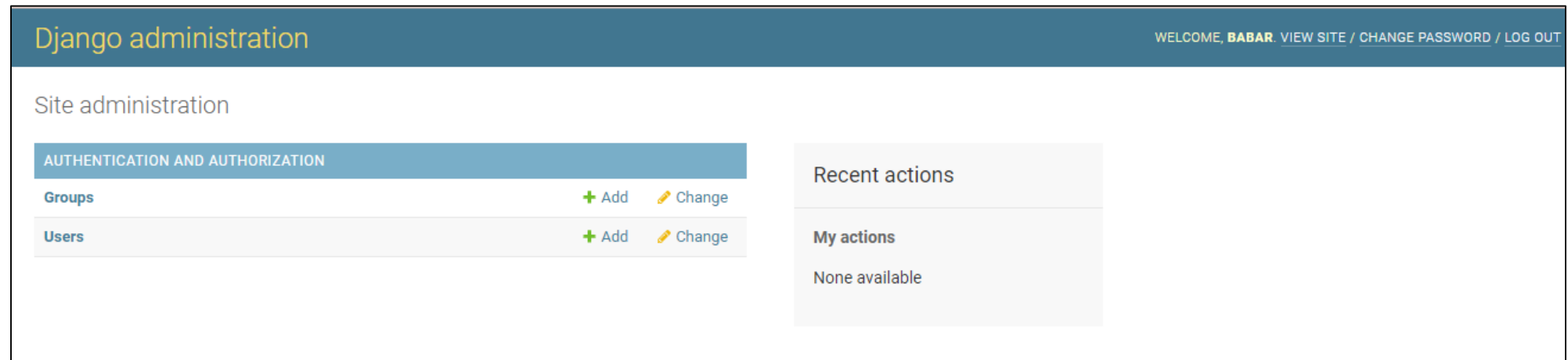
Admin Login (Super User)

- ✓ The following image show that how the super user login to Django Dashboard.
- ✓ Enter Username and Password then click to login.



The image shows the Django administration login interface. It features a dark blue header with the text "Django administration". Below the header, there are two input fields: "Username:" and "Password:". The "Username:" field is a simple text input. The "Password:" field is a text input with a small eye icon to its right, indicating a password field. Below these fields is a blue button labeled "Log in".

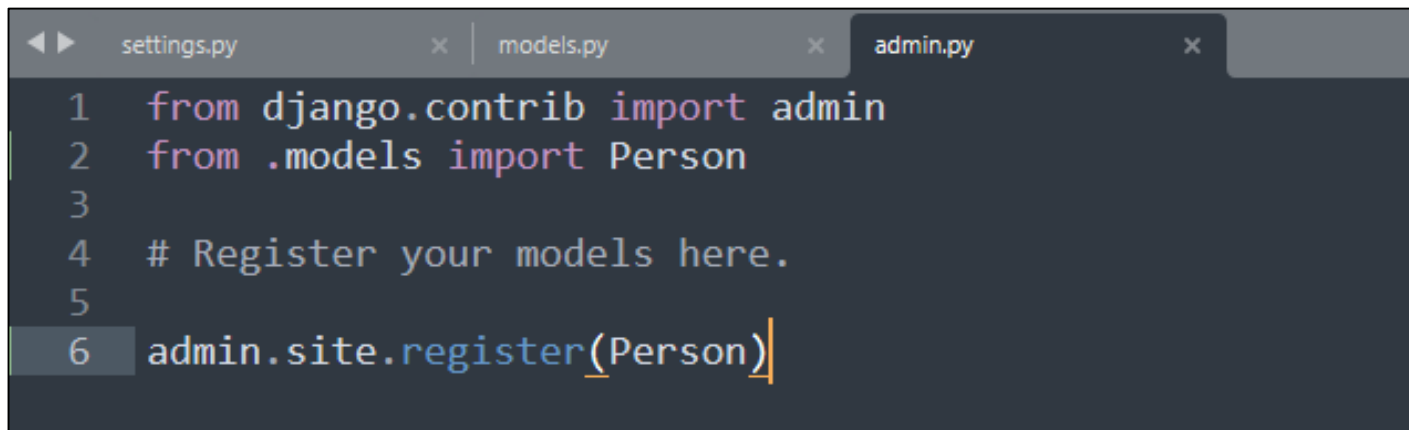
Django Admin Dashboard image



Register Your Model

- ✓ You can see the above image of the admin dashboard. But your model is not showing if you want to show your model in admin site:

Then Go to the “**admin.py**” file and write the following code:



```
1 from django.contrib import admin
2 from .models import Person
3
4 # Register your models here.
5
6 admin.site.register(Person)
```

Here's Your Model

Django administration

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups

[+ Add](#) [Change](#)

Users

[+ Add](#) [Change](#)

FIRST

Persons

[+ Add](#) [Change](#)

Recent actions

My actions

None available

Add Person to Person Model

Home › First › Persons › Add person

Add person

First name:

Last name:

List of All Added Persons

Home › First › Persons

Select person to change ADD PERSON +

Action: 0 of 4 selected

☐	PERSON
☐	Person object (4)
☐	Person object (3)
☐	Person object (2)
☐	Person object (1)

4 persons

Field Option

CharField (and its subclasses) require a **max_length** argument which specifies the size of the **VARCHAR** database field used to store the data.

Null : If True, Django will store empty values as **NULL** in the database. Default is False.

Blank : If True, the field is allowed to be **blank**. Default is False.

If a field has **blank=True**, form validation will allow entry of an empty value.

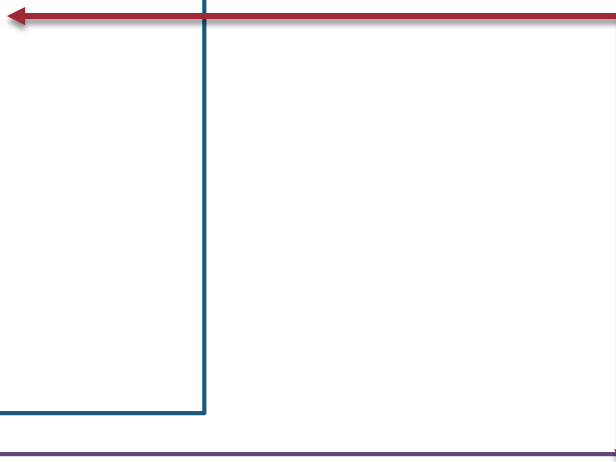
If a field has **blank=False**, the field will be required.

Choices

A **sequence** consisting itself of **iterables** of exactly two items (e.g. [(A, B), (A, B) ...]) to use as choices for this field. If choices are given, they're enforced by **model validation** and the default form widget will be a select box with these choices instead of the standard text field.

```
YEAR_IN_SCHOOL_CHOICES = [  
    ('FR', 'Freshman'),  
    ('SO', 'Sophomore'),  
    ('JR', 'Junior'),  
    ('SR', 'Senior'),  
    ('GR', 'Graduate'),  
]
```

```
year_in_school = models.CharField( max_length=2, choices=YEAR_IN_SCHOOL_CHOICES, default=FRESHMAN)
```



Enumeration Types

In addition, Django provides enumeration types that you can subclass to define choices in a concise way:

```
class Student(models.Model):  
    class YearInSchool(models.TextChoices):  
        FRESHMAN = 'FR', _('Freshman')  
        SOPHOMORE = 'SO', _('Sophomore')  
        JUNIOR = 'JR', _('Junior')  
        SENIOR = 'SR', _('Senior')  
        GRADUATE = 'GR', _('Graduate')
```

```
year_in_school = models.CharField( max_length=2, choices=YearInSchool.choices, default=YearInSchool.FRESHMAN )
```

Field Options

Primary Key

If **True**, this field is the primary key for the model.

If you don't specify **primary_key=True** for any field in your model, Django will automatically add a field to hold the primary key, so you don't need to set **primary_key=True** on any of your fields unless you want to override the default primary-key behavior.

```
person_id = models.IntegerField(primary_key=True)
```


Field Options

unique

If **True**, this field must be unique throughout the table.

This is enforced at the database level and by model validation. If you try to save a model with a duplicate value in a **unique** field, a `django.db.IntegrityError` will be raised by the model's **save()** method.

This option is valid on all field types except **ManyToManyField** and **OneToOneField**.

```
sites = models.ManyToManyField(Site, unique=True)
```

Field Option Unique Example

unique

If True, this field must be unique throughout the table.

Example

```
class Student(models.Model):  
    name = models.CharField(max_length=30, unique=True)
```

Add student

Please correct the error below.

Student with this Name already exists.

Name:

Field Options

Verbose field names

Each field type, except for **ForeignKey**, **ManyToManyField** and **OneToOneField**, takes an optional first positional argument – a verbose name. If the verbose name isn't given, Django will automatically create it using the field's attribute name, converting underscores to spaces.

```
sites = models.ManyToManyField(Site, verbose_name="list of sites")
```

Verbose Field Option Example

Example

```
class Teacher(models.Model):  
    id = models.AutoField(primary_key=True)  
    name = models.CharField("Enter Teacher Name", max_length=30)
```

Add teacher

Enter Teacher Name:



Thanks for Reading